

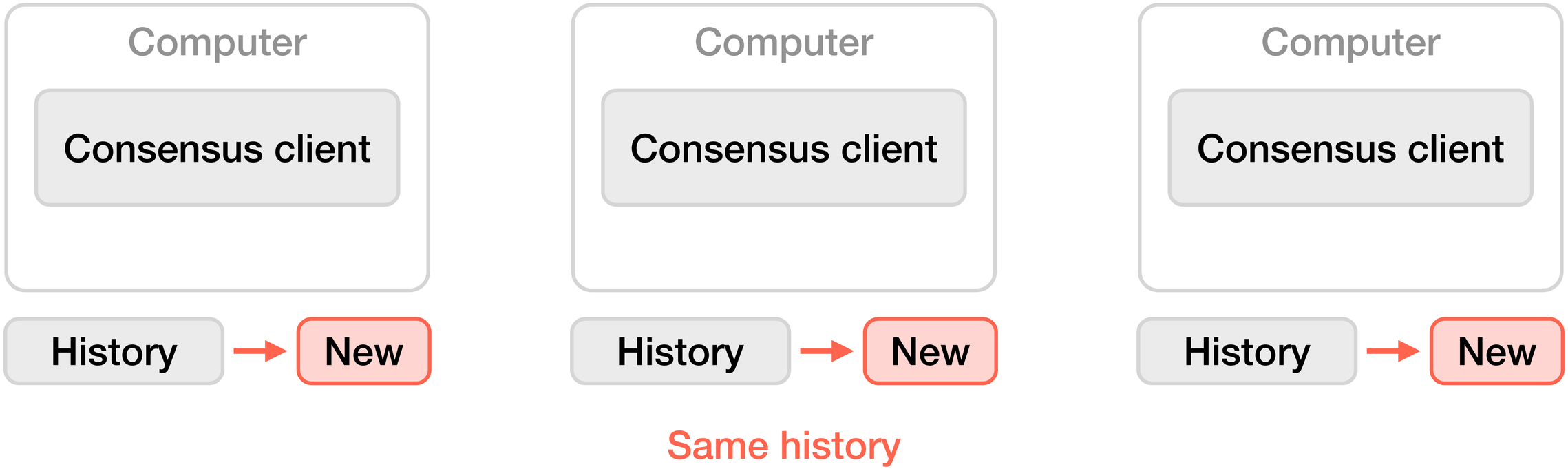


SpecTrum:
Specification-Guided Differential Fuzzing for Ethereum Consensus Clients

Seokhun Jeong[1], Gyeongmin Dan[2], Sukyoung Ryu[1], Sungjae Hwang[2]

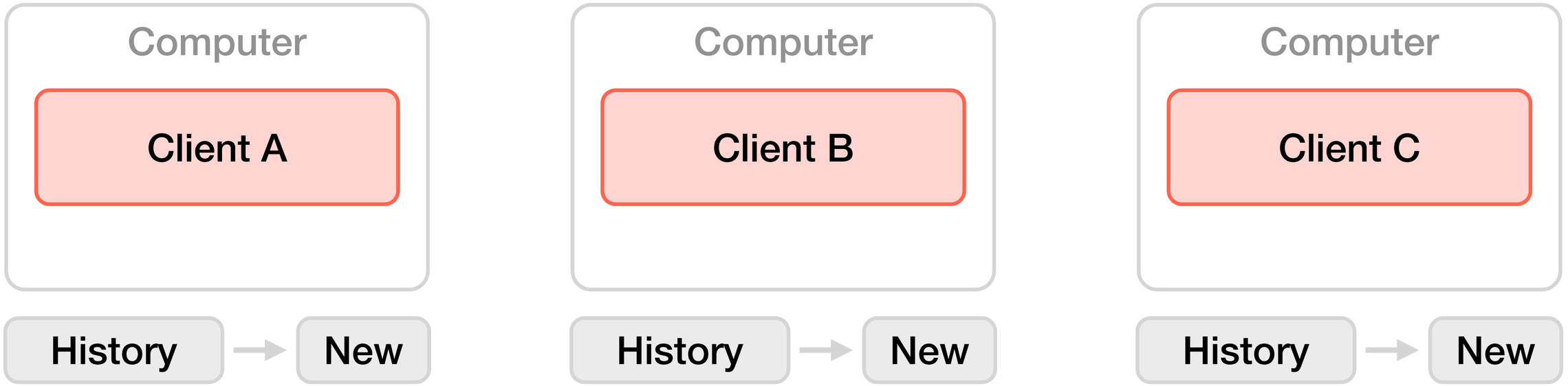
Consensus establishes shared history

Every client must make identical updates to the history



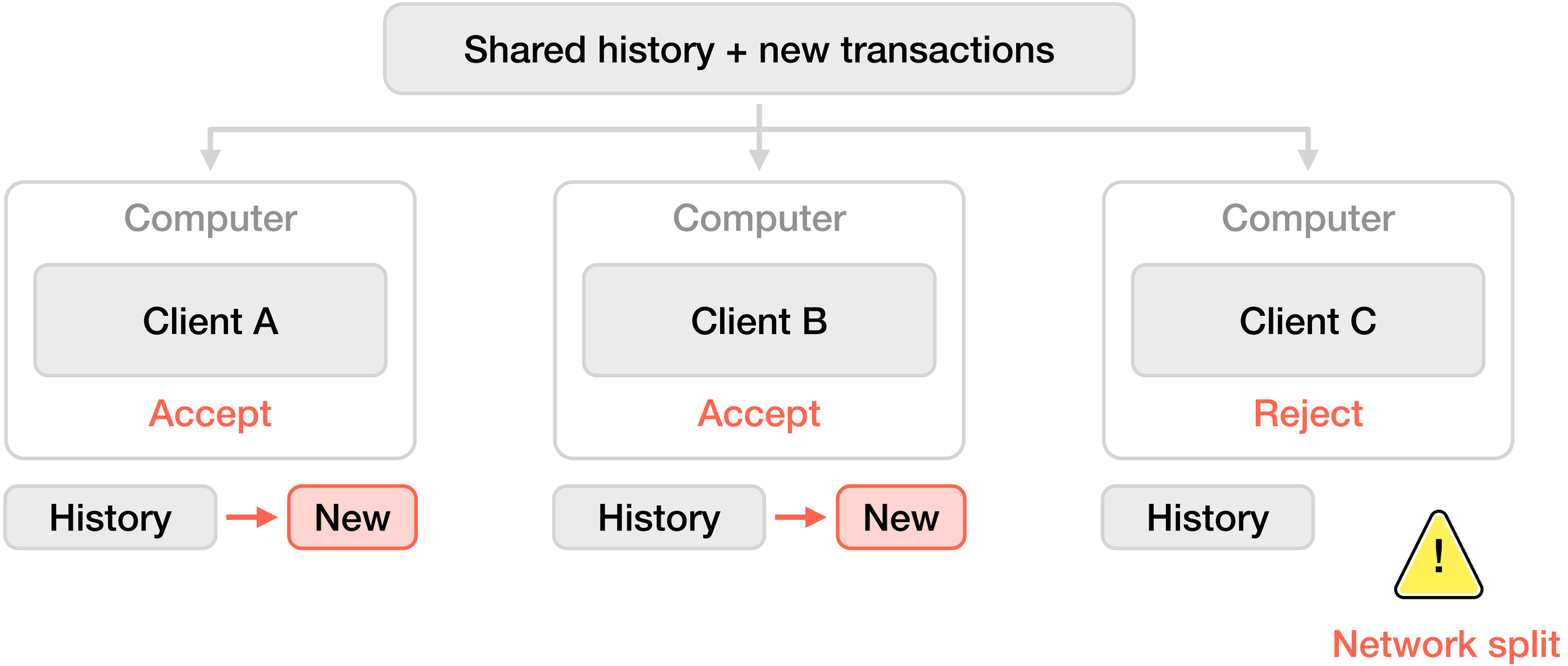
Client diversity limits impact of single failure

Independent implementations of the same rules



Same history

Client disagreements can split the network



State transition gives next state or rejection

Information needed to process blocks,
including validators and balances



Packages new transactions

PySpec defines expected outcomes

state

block

Beacon chain state transition function

The post-state corresponding to a pre-state `state` and a signed block `signed_block` is defined as `state_transition(state, signed_block)`. State transitions that trigger an unhandled exception (e.g. a failed `assert` or an out-of-range list access) are considered invalid. State transitions that cause a `uint64` overflow or underflow are also considered invalid.

```
def state_transition(
    state: BeaconState, signed_block: SignedBeaconBlock, validate_result: b
) -> None:
    block = signed_block.message
    # Process slots (including those with no blocks) since block
    process_slots(state, block.slot)
    # Verify signature
    if validate_result:
        assert verify_block_signature(state, signed_block)
    # Process block
    process_block(state, block)
    # Verify state root
    if validate_result:
        assert block.state_root == hash_tree_root(state)
```

Normal execution

Next state

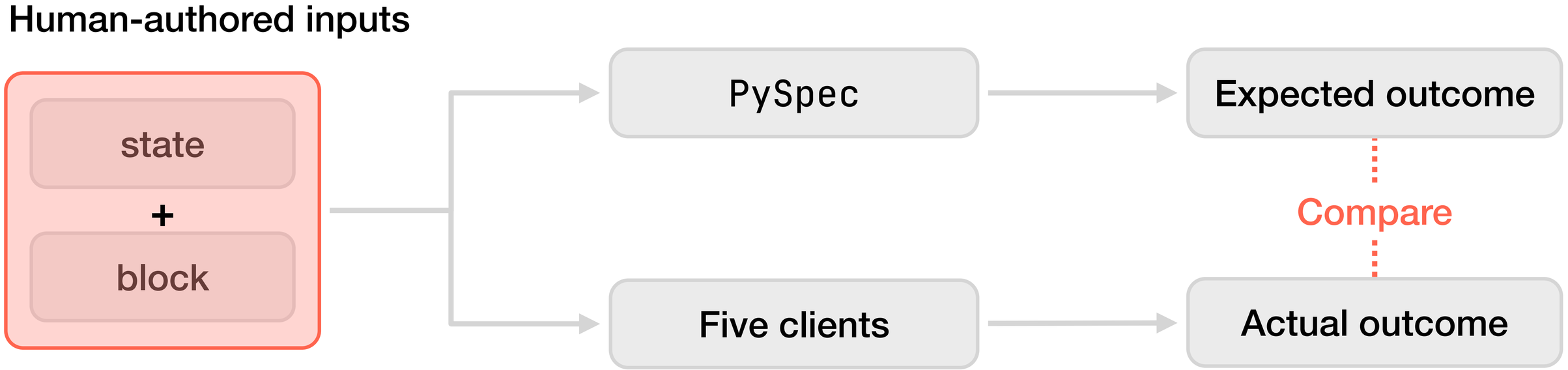
or

Reject input

Python exception

PySpec: official executable Python specification

Official tests compare clients against PySpec



One mutated input

```
balance
→ UINT64_MAX
```

state

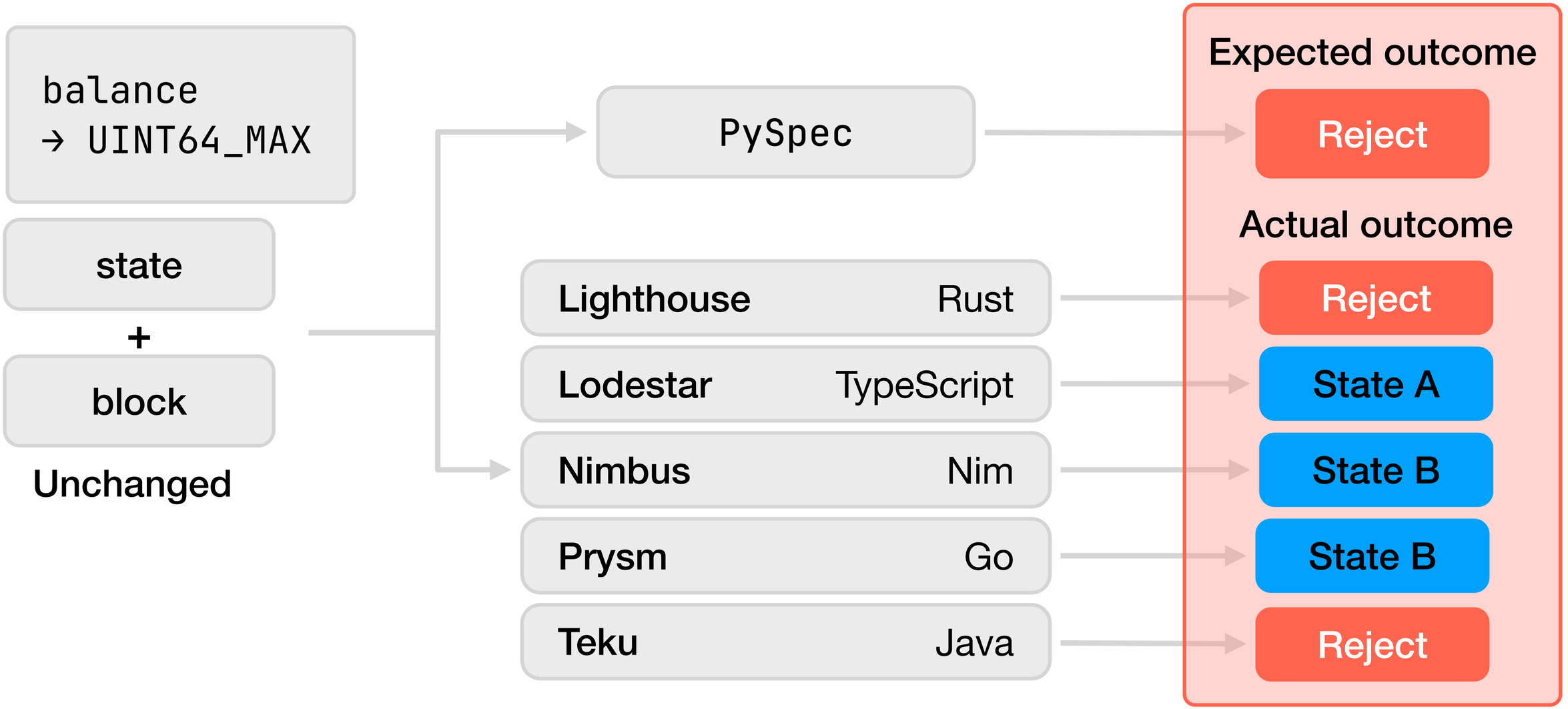
+

block

Unchanged

Problem

One mutated input → client outcome diverges



Implicit exceptions leave testing gaps

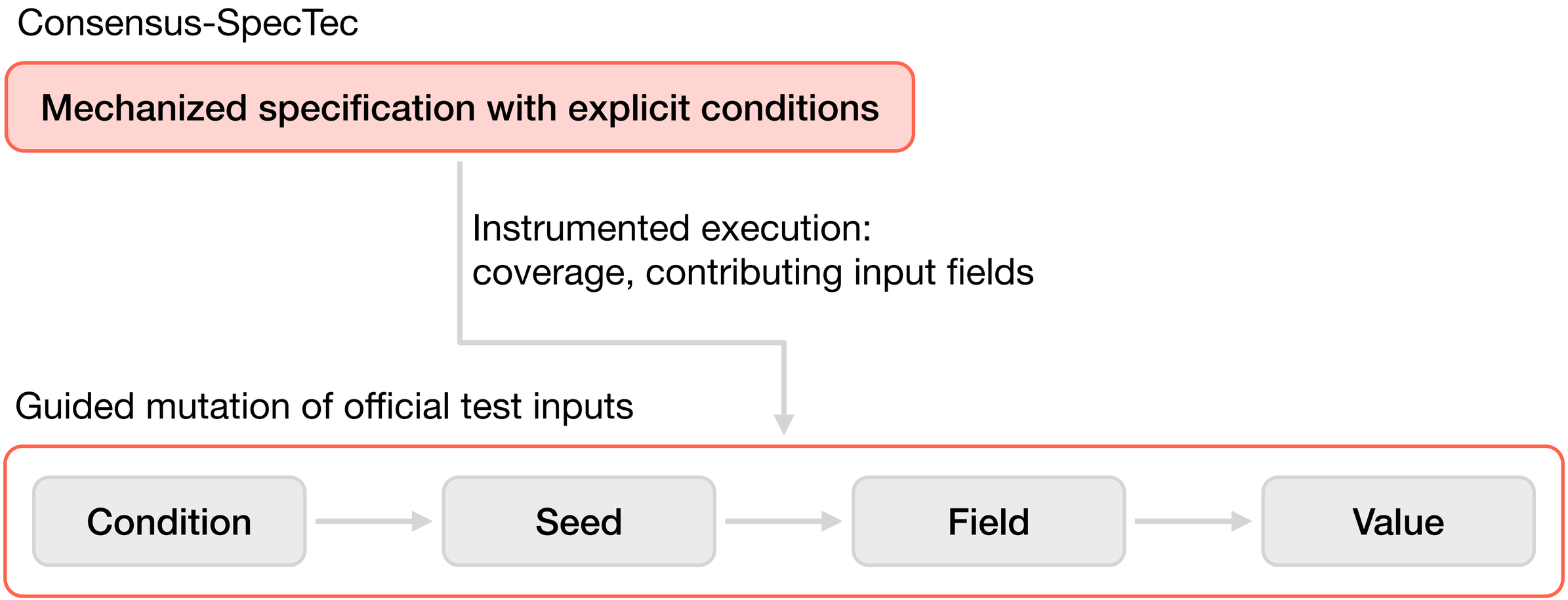
Implicit exceptions: exceptions hidden in operations or type behavior

PySpec

```
if prev_bal * 3 >= total_bal * 2:
```

Expected outcome: uint64 overflow → Python exception → reject input

Mechanized specification guides test generation



Each **execution result** requires different conditions

Example
state update function

```
1 if P:
2     S.x = A
3 if Q:
4     S.x = B
```

S: input state

P, Q: comparisons

A, B: new values for field x

Execution result: return value and state updates

Both outcomes of each branch tested

P	Q	Execution result
true	true	`S[x ← B]`
false	false	`S`

Missing state update: `S[x ← A]`

Each execution result gets **explicit path conditions**

Example
state update function

```
1 if P:
2     S.x = A
3 if Q:
4     S.x = B
```

Manual rewrite

Path condition: when the execution result occurs

Path condition	Execution result
¬P ∧ ¬Q	S
P ∧ ¬Q	S[x ← A]
Q	S[x ← B]

S: input state

P, Q: comparisons

A, B: new values for field x

Each normal execution matches exactly one row (mutually exclusive and exhaustive)

Exception guard conditions make rejection explicit

Comparison P

```
if prev_bal * 3 >= total_bal * 2:
```

Exception guard condition

```
total_bal <= UINT64_MAX / 2
```

true	**false**
multiplication stays within uint64 range	overflow exception, input rejected

Rewritten specification is **executable via P4-SpecTec**

One rule from Consensus-SpecTec (simplified)

```
1 rule Update/x_to_a:
2   S, total_bal, prev_bal, curr_bal
      ~> S[.x = A]

3 -- if total_bal <= UINT64_MAX / 2
4 -- if prev_bal <= UINT64_MAX / 3
5 -- if curr_bal <= UINT64_MAX / 3

6 -- if prev_bal * 3 >= total_bal * 2
7 -- if ~(curr_bal * 3 >= total_bal * 2)
```

Execution result:
$S[x \leftarrow A]$

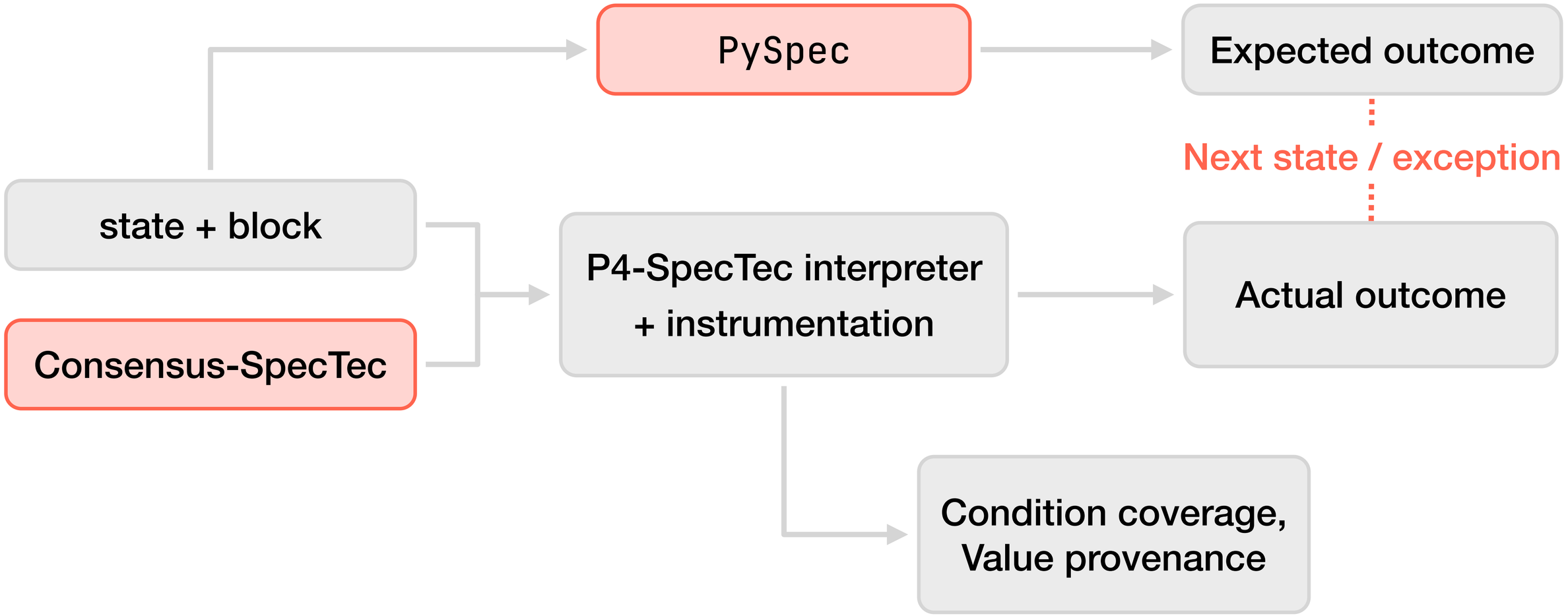
Exception guard conditions

P
¬Q

Path condition

Methodology

Agrees with PySpec on 910 official tests



Condition: Never evaluated to false

```
total_bal <= UINT64_MAX / 2
```

Condition coverage on 581 **state + block** pairs

true	false
21	**0**

Selected as target condition

*Tautologies are manually excluded

Methodology

Seed: Input that reached target condition

```
total_bal <= UINT64_MAX / 2
```

Reached this condition

Condition coverage on 581 **state + block** pairs

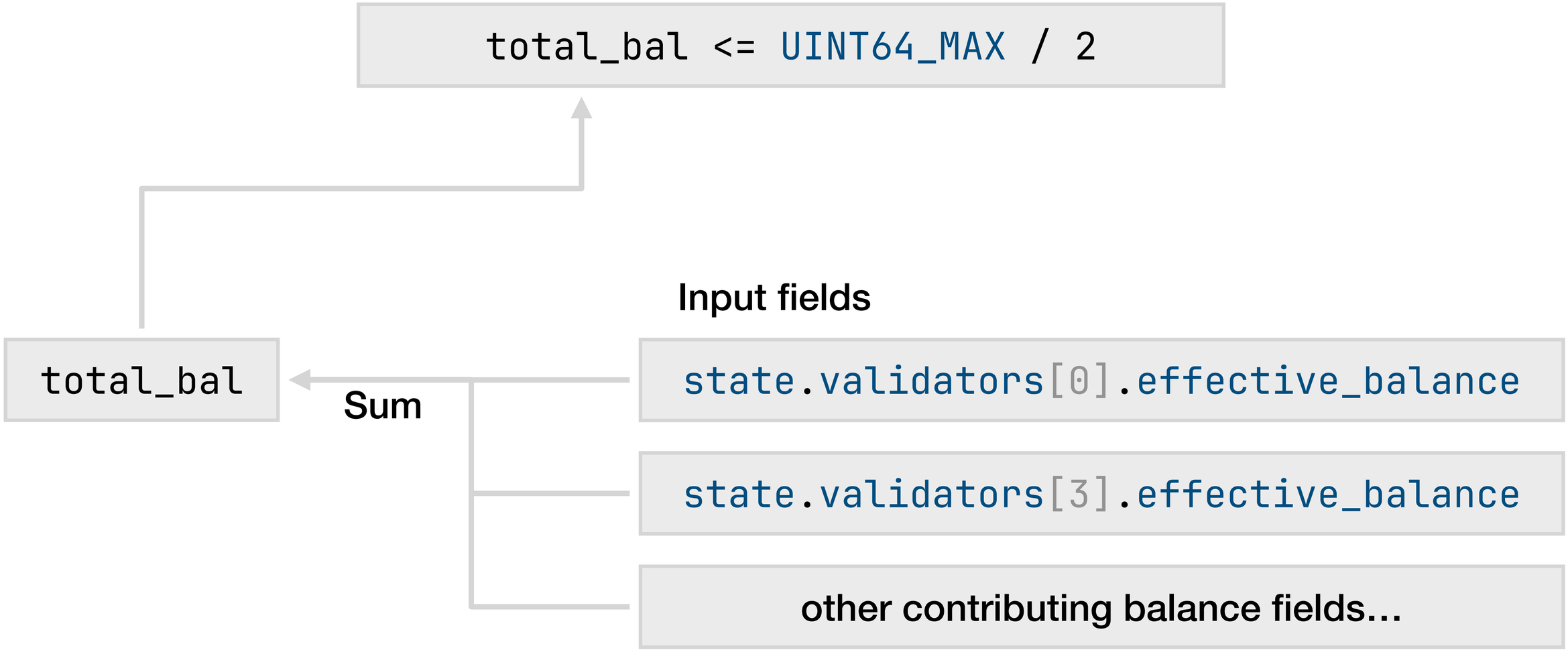
true	false
21	**0**

Seed input
(state + block)

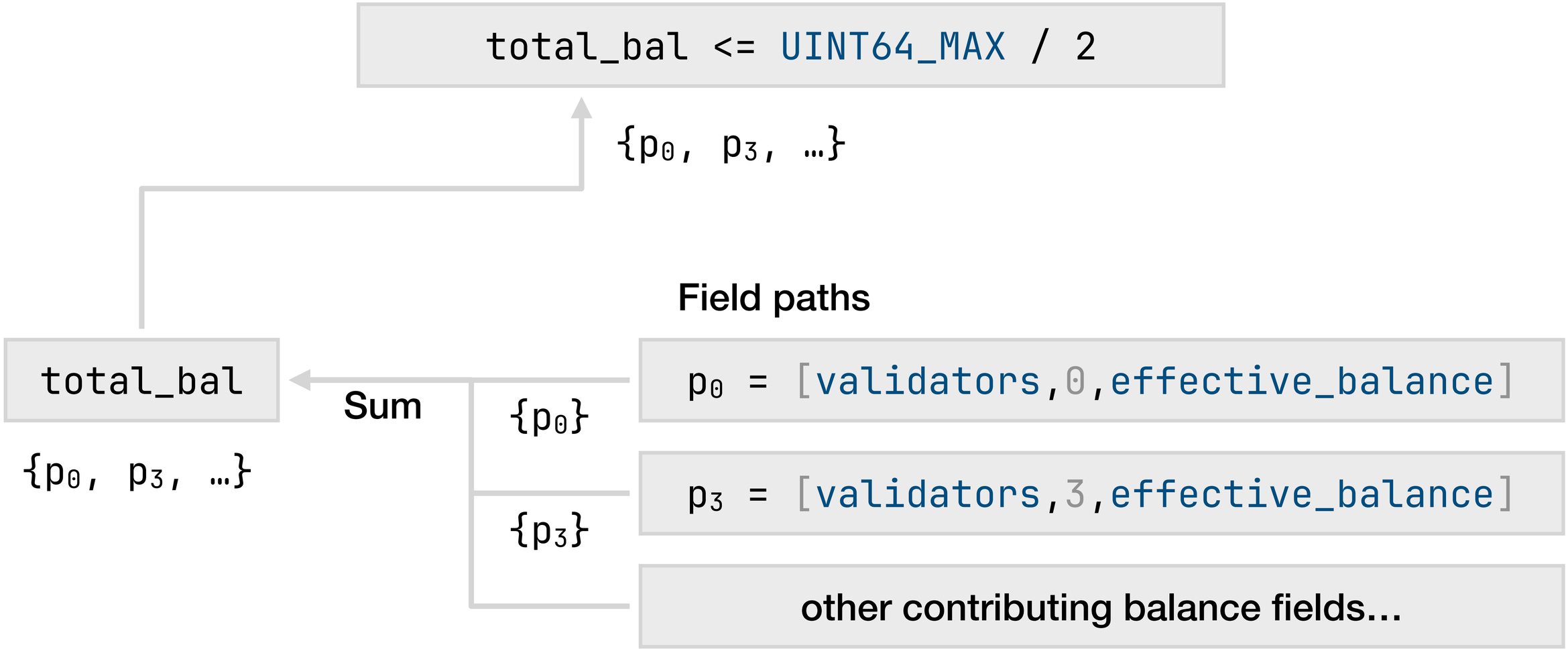
Selected as target condition

*Tautologies are manually excluded

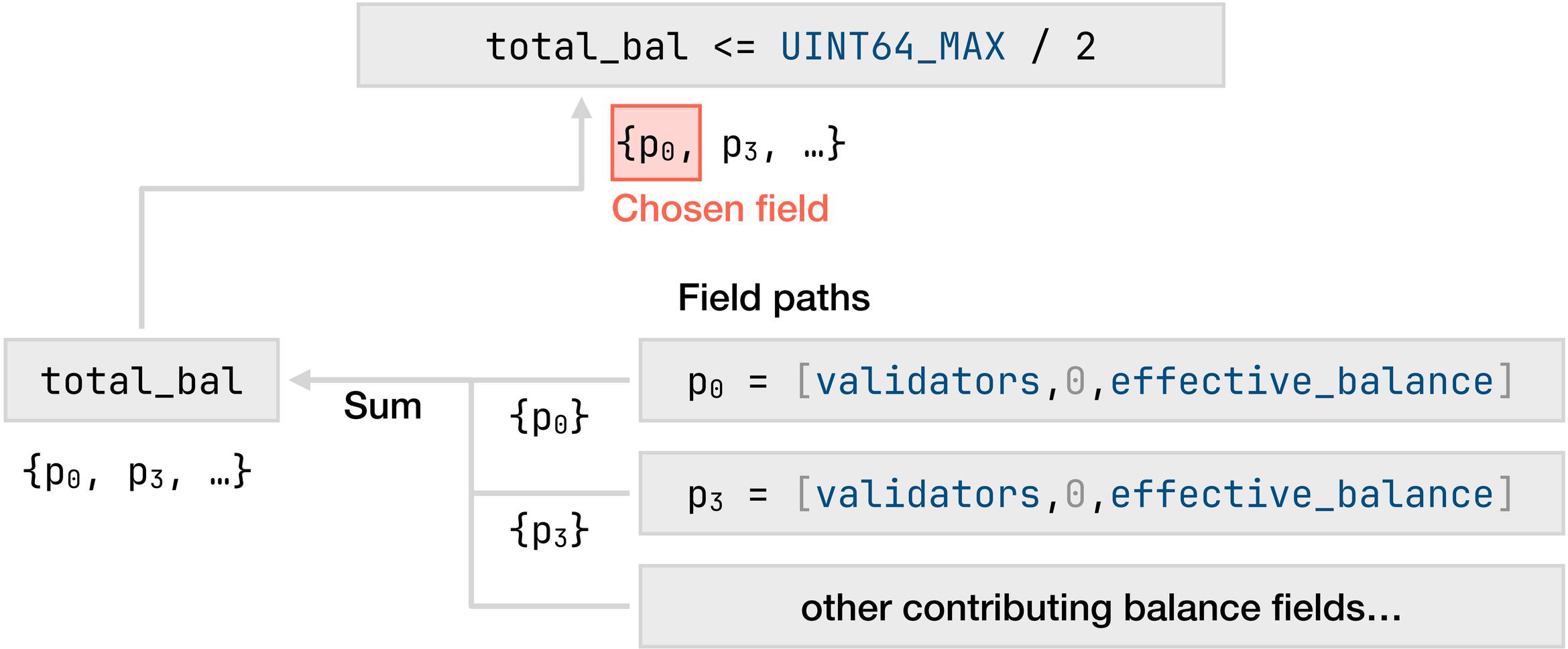
Each value is produced from a set of **input fields**



Input field provenance propagates through execution



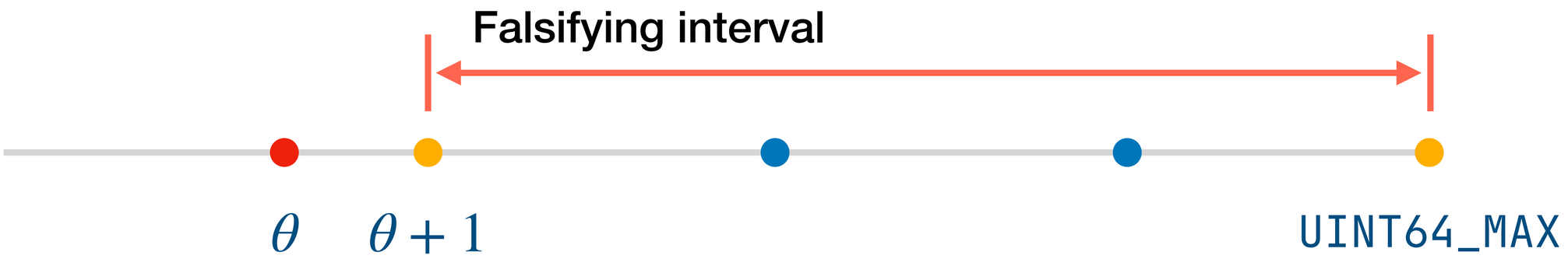
Field: Select from target condition's provenance



Value: Sampled from falsifying interval of condition

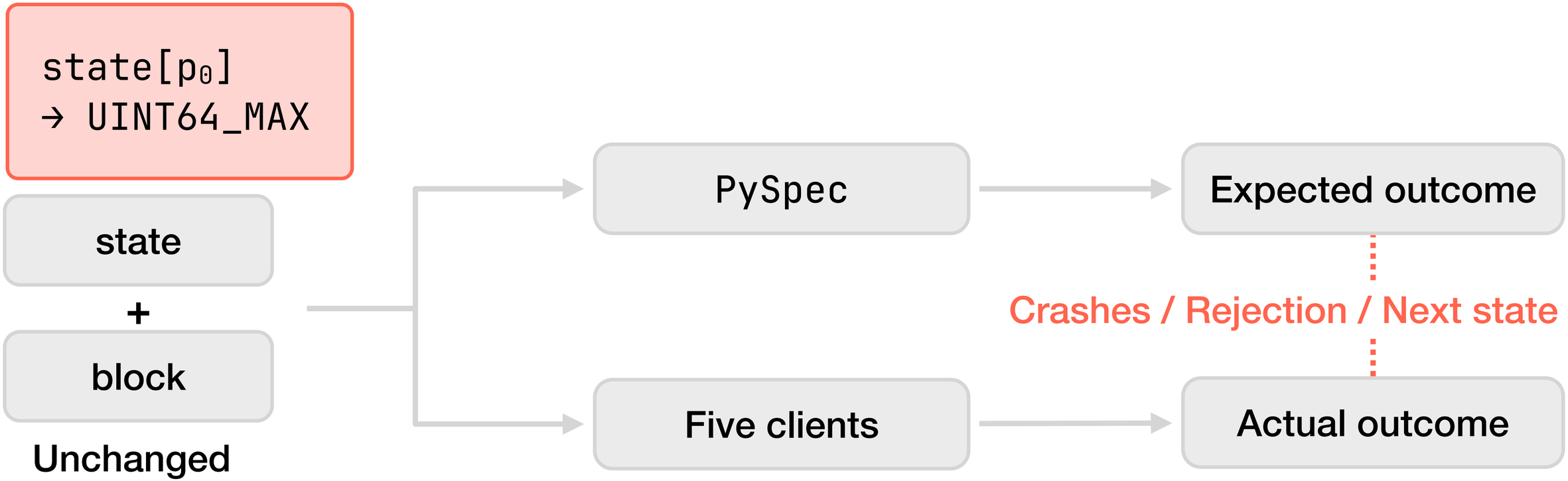
```
total_bal <= UINT64_MAX / 2
```

$\theta = \lfloor \texttt{UINT64_MAX} / 2 \rfloor$



● Boundaries, ● interiors, and ● transition values of interval

Differential testing with mutated inputs



27 divergence cases across five clients

21,444 generated tests across five clients (Capella)

Cases: grouped by root cause and client behavior

Category	Cases	
Input constraint violations	●●●●●●●●●●●	11
Integer range handling	●●●●●●●	7
Integer decoding errors	●●●●	4
Function precondition violations	●●●	3
Protocol version selection errors	●	1
Computation mismatches	●	1
		27

23

block production only

4 cases: also in block validation

4

also in block validation

126 additional false conditions, 2 new branches

Measurement	Official tests	With generated tests	Increase
Conditions evaluated to false	[1] 130/281 (46.3%)	256/281 (91.1%)	+126
PySpec branches covered	152/358	154/358	+2

[1] Denominator excludes conditions that cannot be false

Explicit conditions expose new divergences

65 of 126

Newly falsified conditions

Exception prevention conditions added during rewrite

22 of 27

Divergence cases

Found by targeting exception prevention conditions

More divergences at the same test count

21,444 tests generated by each method

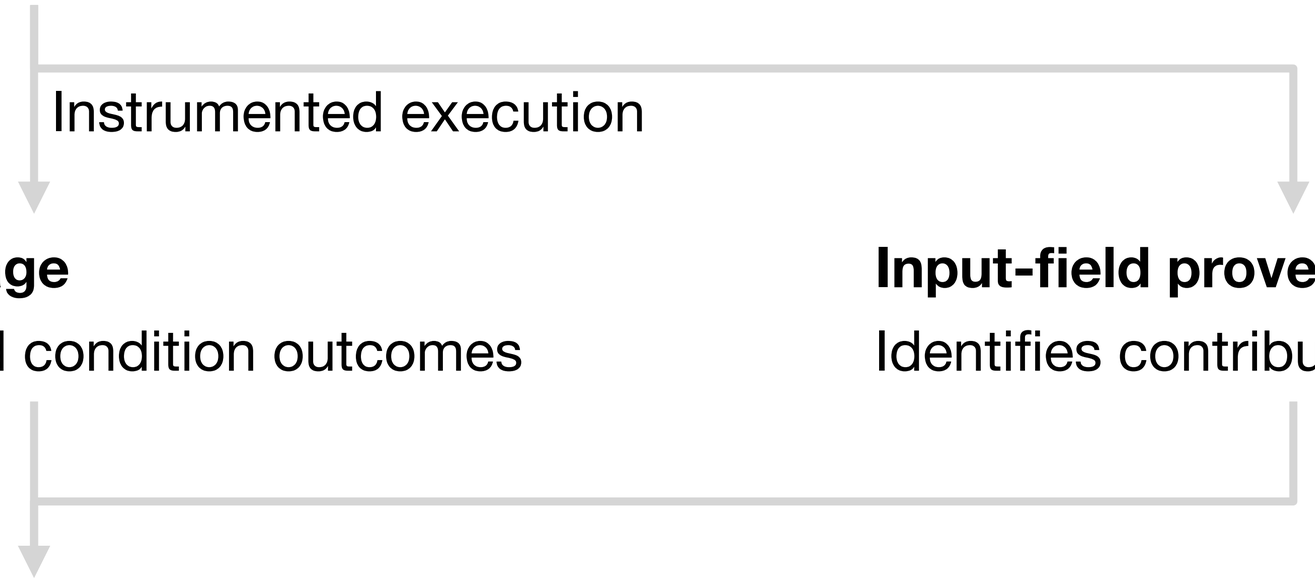
Method	Field and value selection	Cases
Random	Random field and random value	12
Extreme	Random field and boundary value	16
SpecTrum (this work)	Specification conditions and value provenance	27

Random and Extreme each found five cases that SpecTrum missed

Mechanized specifications guide test generation

Consensus-SpecTec

Explicit path conditions and exception guard conditions



Condition coverage

Identifies untested condition outcomes

Input-field provenance

Identifies contributing input fields

Guided mutation of existing inputs